



WEEK 2 INTERNSHIP REPORT

NUMPY AND PANDAS FOR DATA ANALYTICS

Submitted By

ANUSKA GUPTA

B.Tech – Artificial Intelligence and Data Science

KPR Institute of Engineering and Technology

Internship Details

Internship	Domain:	Data	Analytics	/	Python
Week:					2
Duration:	1	Week	(7		Days)
Task: NumPy and Pandas for Data Analytics					
Submission Deadline: 27 September 2026					

INTERNSHIP OVERVIEW

1. About the Week 2 Task

The Week 2 internship task focuses on **NumPy and Pandas for Data Analytics**. The purpose of this task is to develop practical skills for numerical computing, data manipulation, data cleaning, analysis, and exporting processed datasets.

During this week, I worked through a series of practical programs covering NumPy arrays, mathematical and statistical operations, Pandas Data Frames, data inspection, filtering, sorting, missing-value handling, data combination, aggregation, pivot tables, and data export.

The tasks progressively developed into a practical **Student Performance Data Analysis** project using a structured CSV dataset.

2. Objective

- The main objective of this internship task is to understand and apply **NumPy and Pandas** for practical data analytics workflows.

The key objectives are to:

- Understand NumPy arrays and their properties.
- Perform indexing, slicing, and reshaping operations.
- Apply mathematical and statistical operations using NumPy.
- Create and manipulate Pandas Series and Data Frames.
- Read and inspect CSV datasets.
- Select, filter, and sort data.
- Identify and handle missing values.
- Merge and concatenate datasets.
- Perform GroupBy and Pivot Table analysis.
- Export processed data into CSV files.
- Apply the concepts learned through a mini data analysis project.

3. Week 2 Task Coverage

Task	Topic	Status
Task 1	NumPy Introduction & Arrays	Completed
Task 2	Indexing, Slicing & Reshaping	Completed
Task 3	Mathematical & Statistical Operations	Completed
Task 4	Pandas Series & DataFrame	Completed
Task 5	Reading & Inspecting Data	Completed
Task 6	Selecting, Filtering & Sorting	Completed
Task 7	Handling Missing Values	Completed
Task 8	Merge, Concatenate, GroupBy & Pivot Table	Completed
Task 9	Exporting Data	Completed
Task 10	Mini Data Analysis Project	Completed

WEEK 2 LEARNING ROADMAP

From Numerical Computing to Practical Data Analysis

The Week 2 internship activities were structured to progressively develop practical data analytics skills using NumPy and Pandas.

Learning Progression

01 → NumPy Arrays

Understanding one-dimensional and two-dimensional numerical arrays.

↓

02 → Indexing, Slicing & Reshaping

Accessing, extracting, and restructuring array data.

↓

03 → Mathematical & Statistical Operations

Performing calculations and obtaining statistical summaries.

↓

04 → Pandas Series & DataFrame

Creating structured tabular data and manipulating columns.

↓

05 → Reading & Inspecting Data

Loading CSV data and understanding its structure.

↓

06 → Selecting, Filtering & Sorting

Extracting relevant records and organizing data.

↓

07 → Missing Values

Identifying and handling incomplete data.

↓

08 → Merge, GroupBy & Pivot

Combining datasets and generating analytical summaries.

↓

09 → Exporting Data

Saving processed results into reusable CSV files.

↓

10 → Mini Data Analysis Project

Applying the complete workflow to student performance data.

KEY SKILLS PRACTICED

Area	Skills Developed
NumPy	Arrays, indexing, slicing, reshaping
Numerical Computing	Arithmetic and statistical operations
Pandas	Series and DataFrames
Data Inspection	Shape, columns, dtypes, info, describe
Data Selection	Column selection and row filtering
Data Sorting	Ascending and descending sorting
Data Cleaning	Missing-value identification and handling
Data Combination	Merge and concatenation

Data Aggregation	GroupBy and statistical summaries
Data Transformation	Pivot tables and calculated columns
Data Export	CSV generation and verification

WEEK 2 OUTCOME

By the end of Week 2, I was able to use **NumPy and Pandas to load, inspect, clean, manipulate, analyze, summarize, and export structured datasets** through practical data analytics workflows.

TASK 1 — NUMPY INTRODUCTION AND ARRAYS

1. Objective

The objective of this task is to understand the fundamentals of NumPy arrays and their basic properties.

2. Concepts Used

- NumPy library
- One-dimensional arrays
- Two-dimensional arrays
- Array shape
- Array size
- Data type
- `np.array()`

3. Program Description

The program creates a one-dimensional array containing student scores and a two-dimensional score matrix. The shape, size, and data type of each array are displayed.

4. Source Code

File: Task_1_NumPy_Arrays.py

```
import numpy as np

print("=" * 65)
print("          NUMPY & PANDAS FOR DATA ANALYTICS - TASK 1")
print("          NUMPY ARRAYS")
print("=" * 65)

# Creating a one-dimensional NumPy array
student_scores = np.array(
    [72, 85, 91, 68, 77, 88, 95, 81, 74, 89]
)

print("\n1. ONE-DIMENSIONAL ARRAY")
print("-" * 65)

print("Student Scores:")
print(student_scores)

print(f"\nShape      : {student_scores.shape}")
print(f"Size        : {student_scores.size}")
print(f"Data Type   : {student_scores.dtype}")

# Creating a two-dimensional NumPy array
score_matrix = np.array([
    [72, 85, 91, 68, 77],
    [88, 95, 81, 74, 89]
])

print("\n2. TWO-DIMENSIONAL ARRAY")
print("-" * 65)

print("Score Matrix:")
print(score_matrix)
```

```

print(f"\nShape      : {score_matrix.shape}")
print(f"Size        : {score_matrix.size}")
print(f"Data Type : {score_matrix.dtype}")

print("\n" + "-" * 65)
print("NumPy array creation and inspection completed successfully.")
print("=" * 65)

```

5. Program Output

```

• (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_1_NumPy_Arrays.py
=====
          NUMPY & PANDAS FOR DATA ANALYTICS - TASK 1
          NUMPY ARRAYS
=====

1. ONE-DIMENSIONAL ARRAY
-----
Student Scores:
[72 85 91 68 77 88 95 81 74 89]

Shape      : (10,)
Size       : 10
Data Type  : int64

2. TWO-DIMENSIONAL ARRAY
-----
Score Matrix:
[[72 85 91 68 77]
 [88 95 81 74 89]]

Shape      : (2, 5)
Size       : 10
Data Type  : int64

-----
NumPy array creation and inspection completed successfully.
=====
❖ (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics>

```

Figure 1: Task 1 — NUMPY ARRAYS PROGRAM OUTPUT

6. Result

The program successfully created and inspected one-dimensional and two-dimensional NumPy arrays. The shape, size, and data type of the arrays were successfully identified.

TASK 2 — INDEXING, SLICING & RESHAPING

1. Objective

To understand how NumPy arrays can be accessed, sliced, and reshaped for efficient numerical data manipulation.

2. Concepts Used

- Array indexing
- Negative indexing
- Array slicing
- Row access
- Column access
- Reshaping
- Two-dimensional arrays

3. Program Description

The program accesses individual elements from a one-dimensional array, extracts selected portions using slicing, converts the array into a two-dimensional structure, and demonstrates row and column access.

4. Source Code

File: Task_2_Indexing_Slicing_Reshaping.py

```
import numpy as np

print("=" * 70)
print("      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 2")
print("      INDEXING, SLICING & RESHAPING")
print("=" * 70)
```

```

# -----
# 1. ONE-DIMENSIONAL ARRAY
# -----

scores = np.array([72, 85, 91, 68, 77, 88, 95, 81, 74, 89])

print("\n1. ORIGINAL ONE-DIMENSIONAL ARRAY")
print("-" * 70)
print(scores)

# -----
# 2. INDEXING
# -----

print("\n2. INDEXING")
print("-" * 70)

print(f"First element      : {scores[0]}")
print(f"Third element       : {scores[2]}")
print(f>Last element        : {scores[-1]}")

# -----
# 3. SLICING
# -----

print("\n3. SLICING")
print("-" * 70)

print(f"Elements 2 to 5       : {scores[1:5]}")
print(f"First five elements   : {scores[:5]}")
print(f>Last three elements    : {scores[-3:]}")

# -----
# 4. TWO-DIMENSIONAL ARRAY
# -----

score_matrix = scores.reshape(2, 5)

```

```

print("\n4. TWO-DIMENSIONAL ARRAY")
print("-" * 70)
print(score_matrix)

# -----
# 5. ROW AND COLUMN ACCESS
# -----

print("\n5. ROW AND COLUMN ACCESS")
print("-" * 70)

print(f"First row          : {score_matrix[0]}")
print(f"Second row         : {score_matrix[1]}")
print(f"First column         : {score_matrix[:, 0]}")
print(f"Third column          : {score_matrix[:, 2]}")
print(f"Element [1, 3]       : {score_matrix[1, 3]}")

# -----
# 6. RESHAPING
# -----

reshaped_array = scores.reshape(5, 2)

print("\n6. RESHAPING")
print("-" * 70)

print("Original Shape      :", scores.shape)
print("\nReshaped Array (5 × 2):")
print(reshaped_array)

print("\nReshaped Shape       :", reshaped_array.shape)

print("\n" + "-" * 70)
print("Indexing, slicing and reshaping completed successfully.")
print("=" * 70)

```

5. Program Output

```

(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_2_Indexing_Slicing_Reshaping.py
=====
      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 2
      INDEXING, SLICING & RESHAPING
=====

1. ORIGINAL ONE-DIMENSIONAL ARRAY
-----
[72 85 91 68 77 88 95 81 74 89]

2. INDEXING
-----
First element      : 72
Third element      : 91
Last element       : 89

3. SLICING
-----
Elements 2 to 5    : [85 91 68 77]
First five elements : [72 85 91 68 77]
Last three elements : [81 74 89]

4. TWO-DIMENSIONAL ARRAY
-----
[[72 85 91 68 77]
 [88 95 81 74 89]]

5. ROW AND COLUMN ACCESS
-----
First row          : [72 85 91 68 77]
Second row         : [88 95 81 74 89]
First column       : [72 88]
Third column       : [91 81]
Element [1, 3]     : 74

6. RESHAPING
-----
Original Shape      : (10,)

Reshaped Array (5 x 2):
[[72 85]
 [91 68]
 [77 88]
 [95 81]
 [74 89]]

Reshaped Shape      : (5, 2)

-----
Indexing, slicing and reshaping completed successfully.
=====
(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics>

```

Figure 2: Task 2 — Indexing, Slicing & Reshaping Output

6. Result

The program successfully demonstrated NumPy indexing, slicing, row and column access, and array reshaping.

TASK 3: NUMPY MATHEMATICAL & STATISTICAL OPERATIONS

Objective

To perform mathematical and statistical operations on numerical data using NumPy.

Concepts Demonstrated

- Addition
- Subtraction
- Multiplication
- Division
- Mean
- Median
- Minimum
- Maximum
- Standard deviation
- Sum

Implementation

The program performs arithmetic operations on a student-score array and calculates statistical measures to summarize the dataset.

The following functions are used:

```
np.mean()  
np.median()  
np.min()  
np.max()
```

```
np.std()
np.sum()
```

Source Code

File: Task_3_NumPy_Math_Statistics.py

```
import numpy as np

print("=" * 70)
print("      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 3")
print("      MATHEMATICAL & STATISTICAL OPERATIONS")
print("=" * 70)

# Numerical dataset
scores = np.array([72, 85, 91, 68, 77, 88, 95, 81, 74, 89])

print("\nDATASET")
print("-" * 70)
print("Student Scores:")
print(scores)

# -----
# 1. MATHEMATICAL OPERATIONS
# -----

print("\n1. MATHEMATICAL OPERATIONS")
print("-" * 70)

addition = scores + 5
subtraction = scores - 5
multiplication = scores * 2
division = scores / 2

print("Original Values :", scores)
print("Addition (+5)   :", addition)
print("Subtraction (-5):", subtraction)
```

```

print("Multiplication :", multiplication)
print("Division (/2) :", division)

# -----
# 2. STATISTICAL OPERATIONS
# -----

print("\n2. STATISTICAL OPERATIONS")
print("-" * 70)

mean_value = np.mean(scores)
median_value = np.median(scores)
minimum_value = np.min(scores)
maximum_value = np.max(scores)
standard_deviation = np.std(scores)
total_value = np.sum(scores)

print(f"Mean                : {mean_value:.2f}")
print(f"Median                : {median_value:.2f}")
print(f"Minimum                : {minimum_value}")
print(f"Maximum                : {maximum_value}")
print(f"Standard Deviation     : {standard_deviation:.2f}")
print(f"Sum                    : {total_value}")

# -----
# 3. SUMMARY
# -----

print("\n3. DATASET SUMMARY")
print("-" * 70)

print(f"Number of Scores      : {scores.size}")
print(f"Average Score         : {mean_value:.2f}")
print(f"Highest Score         : {maximum_value}")
print(f"Lowest Score          : {minimum_value}")

print("\n" + "-" * 70)
print("Mathematical and statistical operations completed successfully.")
print("=" * 70)

```

Program Output

```
• (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_3_NumPy_Math_Statistics.py
=====
      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 3
      MATHEMATICAL & STATISTICAL OPERATIONS
=====

DATASET
-----
Student Scores:
[72 85 91 68 77 88 95 81 74 89]

1. MATHEMATICAL OPERATIONS
-----
Original Values : [72 85 91 68 77 88 95 81 74 89]
Addition (+5)   : [ 77  90  96  73  82  93 100  86  79  94]
Subtraction (-5): [67 80 86 63 72 83 90 76 69 84]
Multiplication  : [144 170 182 136 154 176 190 162 148 178]
Division (/2)   : [36.  42.5 45.5 34.  38.5 44.  47.5 40.5 37.  44.5]

2. STATISTICAL OPERATIONS
-----
Mean           : 82.00
Median         : 83.00
Minimum        : 68
Maximum        : 95
Standard Deviation : 8.54
Sum            : 820

3. DATASET SUMMARY
-----
Number of Scores : 10
Average Score    : 82.00
Highest Score    : 95
Lowest Score     : 68

-----
Mathematical and statistical operations completed successfully.
=====
• (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> |
```

Figure 3: Task 3 — Mathematical & Statistical Operations

Result

The program successfully performed mathematical and statistical operations. The dataset contained a total score of **820**, with an average of **82.00** and a median of **83.00**.

TASK 4 — PANDAS SERIES AND DATAFRAME

1. Objective

To understand Pandas Series and DataFrames and perform basic DataFrame manipulation.

2. Concepts Demonstrated

- Pandas Series
- Pandas DataFrame
- DataFrame columns
- DataFrame index
- Adding a new column
- Lambda function

3. Implementation

A Pandas Series containing student marks is created first. A DataFrame containing student ID, name, branch, and marks is then created.

A **Status** column is added based on the marks using a conditional lambda function.

4. Source Code

File: Task_4_Pandas_Series_DataFrame.py

```
import pandas as pd

print("=" * 70)
print("      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 4")
```

```

print("                PANDAS SERIES & DATAFRAME")
print("=" * 70)

# -----
# 1. PANDAS SERIES
# -----

print("\n1. PANDAS SERIES")
print("-" * 70)

scores = pd.Series(
    [72, 85, 91, 68, 77],
    index=["S101", "S102", "S103", "S104", "S105"],
    name="Marks"
)

print(scores)

# -----
# 2. CREATE A DATAFRAME
# -----

print("\n2. STUDENT DATAFRAME")
print("-" * 70)

student_data = {
    "Student_ID": ["S101", "S102", "S103", "S104", "S105"],
    "Name": ["Student A", "Student B", "Student C",
             "Student D", "Student E"],
    "Branch": ["AI & DS", "CSE", "AI & DS", "ECE", "CSE"],
    "Marks": [72, 85, 91, 68, 77]
}

df = pd.DataFrame(student_data)

print(df)

# -----
# 3. DISPLAY COLUMN NAMES

```

```
# -----

print("\n3. COLUMN NAMES")
print("-" * 70)

print(list(df.columns))

# -----
# 4. DISPLAY INDEX
# -----

print("\n4. DATAFRAME INDEX")
print("-" * 70)

print(df.index)

# -----
# 5. ADD A NEW COLUMN
# -----

print("\n5. ADDING A NEW COLUMN")
print("-" * 70)

df["Status"] = df["Marks"].apply(
    lambda marks: "Pass" if marks >= 50 else "Fail"
)

print(df)

print("\n" + "-" * 70)
print("Pandas Series and DataFrame operations completed successfully.")
print("=" * 70)
```

5. Program Output

```
• (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_4_Pandas_Series_DataFrame.py
=====
          NUMPY & PANDAS FOR DATA ANALYTICS - TASK 4
          PANDAS SERIES & DATAFRAME
=====

1. PANDAS SERIES
-----
S101    72
S102    85
S103    91
S104    68
S105    77
Name: Marks, dtype: int64

2. STUDENT DATAFRAME
-----
   Student_ID  Name  Branch  Marks
0         S101 Student A  AI & DS    72
1         S102 Student B    CSE    85
2         S103 Student C  AI & DS    91
3         S104 Student D    ECE    68
4         S105 Student E    CSE    77

3. COLUMN NAMES
-----
['Student_ID', 'Name', 'Branch', 'Marks']

4. DATAFRAME INDEX
-----
RangeIndex(start=0, stop=5, step=1)

5. ADDING A NEW COLUMN
-----
   Student_ID  Name  Branch  Marks  Status
0         S101 Student A  AI & DS    72   Pass
1         S102 Student B    CSE    85   Pass
2         S103 Student C  AI & DS    91   Pass
3         S104 Student D    ECE    68   Pass
4         S105 Student E    CSE    77   Pass

-----
Pandas Series and DataFrame operations completed successfully.
=====
```

6. Result

The program successfully created a Pandas Series and DataFrame and demonstrated basic DataFrame manipulation by adding a calculated status column.

TASK 5 — READING AND INSPECTING DATA

1. Objective

To learn how to read a CSV dataset using Pandas and inspect its structure and statistical properties.

2. Concepts Demonstrated

- `pd.read_csv()`
- `head()`
- `tail()`
- `shape`
- `columns`
- `dtypes`
- `info()`
- `describe()`

3. Implementation

The program loads `students.csv` into a Pandas DataFrame and performs an initial inspection of the dataset.

The dataset contains student information including:

- Student ID
- Name
- Branch
- Gender

- Maths
- Reading
- Writing

4. Source Code

File: Task_5_Read_Inspect_Data.py

```
"""
Week 2 Internship: NumPy & Pandas for Data Analytics
Task 5: Reading & Inspecting Data

This program demonstrates how to read a CSV file
and inspect its structure using Pandas.
"""

import pandas as pd

print("=" * 70)
print("          NUMPY & PANDAS FOR DATA ANALYTICS - TASK 5")
print("          READING & INSPECTING DATA")
print("=" * 70)

# -----
# 1. READ CSV FILE
# -----

file_path = "datasets/students.csv"

df = pd.read_csv(file_path)

print("\n1. DATASET LOADED SUCCESSFULLY")
print("-" * 70)
print(df)

# -----
# 2. DISPLAY FIRST FIVE ROWS
# -----

print("\n2. FIRST FIVE ROWS")
```

```

print("-" * 70)
print(df.head())

# -----
# 3. DISPLAY LAST FIVE ROWS
# -----

print("\n3. LAST FIVE ROWS")
print("-" * 70)
print(df.tail())

# -----
# 4. DATASET SHAPE
# -----

print("\n4. DATASET SHAPE")
print("-" * 70)
print(f"Rows      : {df.shape[0]}")
print(f"Columns   : {df.shape[1]}")
print(f"Shape      : {df.shape}")

# -----
# 5. COLUMN NAMES
# -----

print("\n5. COLUMN NAMES")
print("-" * 70)
print(list(df.columns))

# -----
# 6. DATA TYPES
# -----

print("\n6. DATA TYPES")
print("-" * 70)
print(df.dtypes)

# -----
# 7. DATASET INFORMATION
# -----

print("\n7. DATASET INFORMATION")

```

```

print("-" * 70)
df.info()

# -----
# 8. STATISTICAL SUMMARY
# -----

print("\n8. STATISTICAL SUMMARY")
print("-" * 70)
print(df.describe())

print("\n" + "-" * 70)
print("Reading and inspecting the dataset completed successfully.")
print("=" * 70)

```

5. Program Output

```

(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_5_Read_Inspect_Data.py
=====
          NUMPY & PANDAS FOR DATA ANALYTICS - TASK 5
          READING & INSPECTING DATA
=====

1. DATASET LOADED SUCCESSFULLY
-----
   Student_ID  Name  Branch  Gender  Maths  Reading  Writing
0      S101  Student A  AI & DS  Female    72      78      75
1      S102  Student B    CSE    Male    85      82      88
2      S103  Student C  AI & DS  Female    91      90      94
3      S104  Student D    ECE    Male    68      70      65
4      S105  Student E    CSE  Female    77      80      76
5      S106  Student F  AI & DS    Male    88      85      90
6      S107  Student G    ECE  Female    95      92      96
7      S108  Student H    CSE    Male    81      79      84
8      S109  Student I  AI & DS  Female    74      76      72
9      S110  Student J    ECE    Male    89      87      91

2. FIRST FIVE ROWS
-----
   Student_ID  Name  Branch  Gender  Maths  Reading  Writing
0      S101  Student A  AI & DS  Female    72      78      75
1      S102  Student B    CSE    Male    85      82      88
2      S103  Student C  AI & DS  Female    91      90      94
3      S104  Student D    ECE    Male    68      70      65
4      S105  Student E    CSE  Female    77      80      76

3. LAST FIVE ROWS
-----
   Student_ID  Name  Branch  Gender  Maths  Reading  Writing
5      S106  Student F  AI & DS    Male    88      85      90
6      S107  Student G    ECE  Female    95      92      96
7      S108  Student H    CSE    Male    81      79      84
8      S109  Student I  AI & DS  Female    74      76      72
9      S110  Student J    ECE    Male    89      87      91

```

4. DATASET SHAPE

```
-----  
Rows      : 10  
Columns   : 7  
Shape     : (10, 7)
```

5. COLUMN NAMES

```
-----  
['Student_ID', 'Name', 'Branch', 'Gender', 'Maths', 'Reading', 'Writing']
```

6. DATA TYPES

```
-----  
Student_ID      str  
Name             str  
Branch          str  
Gender          str  
Maths           int64  
Reading         int64  
Writing         int64  
dtype: object
```

7. DATASET INFORMATION

```
-----  
<class 'pandas.DataFrame'>  
RangeIndex: 10 entries, 0 to 9  
Data columns (total 7 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Student_ID  10 non-null    str  
1   Name        10 non-null    str  
2   Branch      10 non-null    str  
3   Gender      10 non-null    str  
4   Maths       10 non-null    int64  
5   Reading     10 non-null    int64  
6   Writing     10 non-null    int64  
dtypes: int64(3), str(4)  
memory usage: 692.0 bytes
```

```
(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_5_Read_Inspect_Data.py
=====
NUMPY & PANDAS FOR DATA ANALYTICS - TASK 5
READING & INSPECTING DATA
=====

1. DATASET LOADED SUCCESSFULLY
-----
  Student_ID      Name  Branch  Gender  Maths  Reading  Writing
0      S101  Student A  AI & DS  Female    72      78      75
1      S102  Student B    CSE    Male    85      82      88
2      S103  Student C  AI & DS  Female    91      90      94
3      S104  Student D    ECE    Male    68      70      65
4      S105  Student E    CSE    Female   77      80      76
5      S106  Student F  AI & DS    Male    88      85      90
6      S107  Student G    ECE    Female   95      92      96
7      S108  Student H    CSE    Male    81      79      84
8      S109  Student I  AI & DS  Female    74      76      72
9      S110  Student J    ECE    Male    89      87      91

2. FIRST FIVE ROWS
-----
  Student_ID      Name  Branch  Gender  Maths  Reading  Writing
0      S101  Student A  AI & DS  Female    72      78      75
1      S102  Student B    CSE    Male    85      82      88
2      S103  Student C  AI & DS  Female    91      90      94
3      S104  Student D    ECE    Male    68      70      65
4      S105  Student E    CSE    Female   77      80      76

3. LAST FIVE ROWS
-----
  Student_ID      Name  Branch  Gender  Maths  Reading  Writing
5      S106  Student F  AI & DS    Male    88      85      90
6      S107  Student G    ECE    Female   95      92      96
7      S108  Student H    CSE    Male    81      79      84
8      S109  Student I  AI & DS  Female    74      76      72
9      S110  Student J    ECE    Male    89      87      91
```

6. Result

The student dataset was successfully loaded and inspected. Its dimensions, column names, data types, structure, and statistical summary were obtained.

TASK 6 — SELECTING, FILTERING AND SORTING DATA

1. Objective

To learn how to select specific columns, filter records based on conditions, and sort data using Pandas.

2. Concepts Demonstrated

- Single-column selection
- Multiple-column selection
- Conditional filtering
- Multiple conditions
- Ascending sorting
- Descending sorting
- Top-record selection

3. Implementation

The program selects specific columns from the student dataset and filters students based on their Maths scores and branch.

It also sorts students by Maths scores and identifies the top three students.

4. Source Code

File: Task_6_Filter_Sort_Data.py

```
import pandas as pd

print("=" * 70)
print("      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 6")
print("      SELECTING, FILTERING & SORTING DATA")
print("=" * 70)

# -----
# 1. READ DATASET
# -----

file_path = "datasets/students.csv"

df = pd.read_csv(file_path)

print("\n1. ORIGINAL DATASET")
print("-" * 70)
```

```
print(df)

# -----
# 2. SELECT A SINGLE COLUMN
# -----

print("\n2. SELECTING A SINGLE COLUMN")
print("-" * 70)

print("Student Names:")
print(df["Name"])

# -----
# 3. SELECT MULTIPLE COLUMNS
# -----

print("\n3. SELECTING MULTIPLE COLUMNS")
print("-" * 70)

selected_columns = df[["Name", "Branch", "Maths"]]

print(selected_columns)

# -----
# 4. FILTER STUDENTS BASED ON MATHS SCORE
# -----

print("\n4. FILTERING STUDENTS WITH MATHS SCORE >= 80")
print("-" * 70)

high_math_students = df[df["Maths"] >= 80]

print(high_math_students)

# -----
# 5. FILTER STUDENTS BY BRANCH
# -----

print("\n5. FILTERING AI & DS STUDENTS")
print("-" * 70)
```

```

ai_ds_students = df[df["Branch"] == "AI & DS"]

print(ai_ds_students)

# -----
# 6. MULTIPLE CONDITIONS
# -----

print("\n6. FILTERING AI & DS STUDENTS WITH MATHS >= 80")
print("-" * 70)

filtered_students = df[
    (df["Branch"] == "AI & DS") &
    (df["Maths"] >= 80)
]

print(filtered_students)

# -----
# 7. SORT BY MATHS SCORE - ASCENDING
# -----

print("\n7. SORTING BY MATHS SCORE - ASCENDING")
print("-" * 70)

ascending_data = df.sort_values(
    by="Maths",
    ascending=True
)

print(ascending_data)

# -----
# 8. SORT BY MATHS SCORE - DESCENDING
# -----

print("\n8. SORTING BY MATHS SCORE - DESCENDING")
print("-" * 70)

descending_data = df.sort_values(
    by="Maths",
    ascending=False
)

```

```

)

print(descending_data)

# -----
# 9. TOP THREE STUDENTS
# -----

print("\n9. TOP THREE STUDENTS BY MATHS SCORE")
print("-" * 70)

top_three = df.sort_values(
    by="Maths",
    ascending=False
).head(3)

print(top_three[["Student_ID", "Name", "Branch", "Maths"]])

print("\n" + "-" * 70)
print("Selecting, filtering and sorting completed successfully.")
print("=" * 70)

```

5. Program Output

```

(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics>
(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_6_Filter_Sort_Data.py
=====
          NUMPY & PANDAS FOR DATA ANALYTICS - TASK 6
          SELECTING, FILTERING & SORTING DATA
=====

1. ORIGINAL DATASET
-----

```

	Student_ID	Name	Branch	Gender	Maths	Reading	Writing
0	S101	Student A	AI & DS	Female	72	78	75
1	S102	Student B	CSE	Male	85	82	88
2	S103	Student C	AI & DS	Female	91	90	94
3	S104	Student D	ECE	Male	68	70	65
4	S105	Student E	CSE	Female	77	80	76
5	S106	Student F	AI & DS	Male	88	85	90
6	S107	Student G	ECE	Female	95	92	96
7	S108	Student H	CSE	Male	81	79	84
8	S109	Student I	AI & DS	Female	74	76	72
9	S110	Student J	ECE	Male	89	87	91

```

-----
2. SELECTING A SINGLE COLUMN
-----
Student Names:
0    Student A
1    Student B
2    Student C
3    Student D
4    Student E
5    Student F
6    Student G
7    Student H
8    Student I
9    Student J
Name: Name, dtype: str

```

3. SELECTING MULTIPLE COLUMNS

	Name	Branch	Maths
0	Student A	AI & DS	72
1	Student B	CSE	85
2	Student C	AI & DS	91
3	Student D	ECE	68
4	Student E	CSE	77
5	Student F	AI & DS	88
6	Student G	ECE	95
7	Student H	CSE	81
8	Student I	AI & DS	74
9	Student J	ECE	89

4. FILTERING STUDENTS WITH MATHS SCORE >= 80

	Student_ID	Name	Branch	Gender	Maths	Reading	Writing
1	S102	Student B	CSE	Male	85	82	88
2	S103	Student C	AI & DS	Female	91	90	94
5	S106	Student F	AI & DS	Male	88	85	90
6	S107	Student G	ECE	Female	95	92	96
7	S108	Student H	CSE	Male	81	79	84
9	S110	Student J	ECE	Male	89	87	91

5. FILTERING AI & DS STUDENTS

	Student_ID	Name	Branch	Gender	Maths	Reading	Writing
0	S101	Student A	AI & DS	Female	72	78	75
2	S103	Student C	AI & DS	Female	91	90	94
5	S106	Student F	AI & DS	Male	88	85	90
8	S109	Student I	AI & DS	Female	74	76	72

6. FILTERING AI & DS STUDENTS WITH MATHS >= 80

	Student_ID	Name	Branch	Gender	Maths	Reading	Writing
2	S103	Student C	AI & DS	Female	91	90	94
5	S106	Student F	AI & DS	Male	88	85	90

7. SORTING BY MATHS SCORE - ASCENDING

	Student_ID	Name	Branch	Gender	Maths	Reading	Writing
3	S104	Student D	ECE	Male	68	70	65
0	S101	Student A	AI & DS	Female	72	78	75
8	S109	Student I	AI & DS	Female	74	76	72
4	S105	Student E	CSE	Female	77	80	76
7	S108	Student H	CSE	Male	81	79	84
1	S102	Student B	CSE	Male	85	82	88
5	S106	Student F	AI & DS	Male	88	85	90
9	S110	Student J	ECE	Male	89	87	91
2	S103	Student C	AI & DS	Female	91	90	94
6	S107	Student G	ECE	Female	95	92	96

8. SORTING BY MATHS SCORE - DESCENDING

	Student_ID	Name	Branch	Gender	Maths	Reading	Writing
6	S107	Student G	ECE	Female	95	92	96
2	S103	Student C	AI & DS	Female	91	90	94
9	S110	Student J	ECE	Male	89	87	91
5	S106	Student F	AI & DS	Male	88	85	90
1	S102	Student B	CSE	Male	85	82	88
7	S108	Student H	CSE	Male	81	79	84
4	S105	Student E	CSE	Female	77	80	76
8	S109	Student I	AI & DS	Female	74	76	72
0	S101	Student A	AI & DS	Female	72	78	75
3	S104	Student D	ECE	Male	68	70	65

9. TOP THREE STUDENTS BY MATHS SCORE

```
-----  
Student_ID      Name      Branch  Maths  
6      S107  Student G      ECE      95  
2      S103  Student C      AI & DS    91  
9      S110  Student J      ECE      89  
-----
```

Selecting, filtering and sorting completed successfully.

```
=====
```

```
. (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> █
```

6. Result

The program successfully selected relevant columns, filtered students based on specified conditions, sorted the dataset, and identified high-performing students.

TASK 7 — HANDLING MISSING VALUES

1. Objective

To identify, count, remove, and replace missing values in a Pandas DataFrame.

2. Concepts Demonstrated

- `isnull()`
- `sum()`
- `dropna()`
- `fillna()`
- Mean-based imputation

3. Implementation

A student dataset containing intentional missing values was created. The program identifies missing values, calculates their count, removes incomplete rows, and fills missing numerical values using column means.

4. Source Code

File: Task_7_Missing_Values.py

```
import pandas as pd
import numpy as np

print("=" * 70)
print("      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 7")
print("      HANDLING MISSING VALUES")
print("=" * 70)

# -----
# 1. CREATE DATASET WITH MISSING VALUES
# -----

data = {
    "Student_ID": [
        "S101", "S102", "S103", "S104", "S105",
        "S106", "S107", "S108"
    ],
    "Name": [
        "Student A", "Student B", "Student C", "Student D",
        "Student E", "Student F", "Student G", "Student H"
    ],
    "Branch": [
        "AI & DS", "CSE", "AI & DS", "ECE",
        "CSE", "AI & DS", "ECE", "CSE"
    ],
    "Maths": [
        72, 85, np.nan, 68, 77, 88, 95, 81
    ],
    "Reading": [
        78, 82, 90, np.nan, 80, 85, 92, 79
    ],
    "Writing": [
        75, 88, 94, 65, np.nan, 90, 96, 84
    ]
}

df = pd.DataFrame(data)
```

```

print("\n1. DATASET WITH MISSING VALUES")
print("-" * 70)
print(df)

# -----
# 2. IDENTIFY MISSING VALUES
# -----

print("\n2. IDENTIFYING MISSING VALUES")
print("-" * 70)

print(df.isnull())

# -----
# 3. COUNT MISSING VALUES
# -----

print("\n3. COUNT OF MISSING VALUES")
print("-" * 70)

missing_count = df.isnull().sum()

print(missing_count)

# -----
# 4. TOTAL MISSING VALUES
# -----

print("\n4. TOTAL MISSING VALUES")
print("-" * 70)

total_missing = df.isnull().sum().sum()

print(f"Total Missing Values : {total_missing}")

# -----
# 5. DROP ROWS WITH MISSING VALUES
# -----

print("\n5. DATASET AFTER DROPPING MISSING ROWS")

```

```

print("-" * 70)

dropped_df = df.dropna()

print(dropped_df)

# -----
# 6. FILL MISSING NUMERICAL VALUES WITH MEAN
# -----

print("\n6. FILLING MISSING VALUES WITH COLUMN MEAN")
print("-" * 70)

filled_df = df.copy()

numeric_columns = ["Maths", "Reading", "Writing"]

for column in numeric_columns:
    filled_df[column] = filled_df[column].fillna(
        filled_df[column].mean()
    )
print(filled_df)

# -----
# 7. VERIFY MISSING VALUES AFTER FILLING
# -----

print("\n7. MISSING VALUES AFTER FILLING")
print("-" * 70)

print(filled_df.isnull().sum())

print("\n" + "-" * 70)
print("Missing value handling completed successfully.")
print("=" * 70)

```

5. Program Output

```
(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_7_Missing_Values.py
```

```
=====
NUPY & PANDAS FOR DATA ANALYTICS - TASK 7
HANDLING MISSING VALUES
=====
```

1. DATASET WITH MISSING VALUES

```
-----
Student_ID  Name  Branch  Maths  Reading  Writing
0      S101 Student A  AI & DS   72.0    78.0    75.0
1      S102 Student B    CSE   85.0    82.0    88.0
2      S103 Student C  AI & DS    NaN    90.0    94.0
3      S104 Student D    ECE   68.0    NaN    65.0
4      S105 Student E    CSE   77.0    80.0    NaN
5      S106 Student F  AI & DS   88.0    85.0    90.0
6      S107 Student G    ECE   95.0    92.0    96.0
7      S108 Student H    CSE   81.0    79.0    84.0
```

2. IDENTIFYING MISSING VALUES

```
-----
Student_ID  Name  Branch  Maths  Reading  Writing
0      False  False  False  False   False   False
1      False  False  False  False   False   False
2      False  False  False  True    False   False
3      False  False  False  False   True    False
4      False  False  False  False   False   True
5      False  False  False  False   False   False
6      False  False  False  False   False   False
7      False  False  False  False   False   False
```

3. COUNT OF MISSING VALUES

```
-----
Student_ID    0
Name          0
Branch        0
Maths         1
Reading       1
Writing       1
dtype: int64
```

4. TOTAL MISSING VALUES

```
-----
Total Missing Values : 3
```

5. DATASET AFTER DROPPING MISSING ROWS

```
-----  
Student_ID  Name  Branch  Maths  Reading  Writing  
0      S101  Student A  AI & DS  72.0    78.0    75.0  
1      S102  Student B      CSE  85.0    82.0    88.0  
5      S106  Student F  AI & DS  88.0    85.0    90.0  
6      S107  Student G      ECE  95.0    92.0    96.0  
7      S108  Student H      CSE  81.0    79.0    84.0
```

6. FILLING MISSING VALUES WITH COLUMN MEAN

```
-----  
Student_ID  Name  Branch  Maths  Reading  Writing  
0      S101  Student A  AI & DS  72.000000  78.000000  75.000000  
1      S102  Student B      CSE  85.000000  82.000000  88.000000  
2      S103  Student C  AI & DS  80.857143  90.000000  94.000000  
3      S104  Student D      ECE  68.000000  83.714286  65.000000  
4      S105  Student E      CSE  77.000000  80.000000  84.571429  
5      S106  Student F  AI & DS  88.000000  85.000000  90.000000  
6      S107  Student G      ECE  95.000000  92.000000  96.000000  
7      S108  Student H      CSE  81.000000  79.000000  84.000000
```

7. MISSING VALUES AFTER FILLING

```
-----  
Student_ID  0  
Name        0  
Branch      0  
Maths       0  
Reading     0  
Writing     0  
dtype: int64
```

```
-----  
Missing value handling completed successfully.  
=====
```

```
❖ (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> █
```

6. Result

Three missing values were identified in the dataset. Missing numerical values were successfully handled using mean-based imputation.

TASK 8 — MERGE, CONCATENATE, GROUPBY & PIVOT TABLE

1. Objective

To learn how to combine multiple datasets and generate summarized information using Pandas.

2. Concepts Demonstrated

- `pd.merge()`
- `pd.concat()`
- `groupby()`
- `pd.pivot_table()`
- Aggregation using `mean()`

3. Implementation

Two datasets containing student details and performance information were created.

The datasets were merged using `Student_ID`. Additional records were combined using concatenation.

Branch-wise subject averages were then calculated using `GroupBy` and `Pivot Table` operations.

4. Source Code

File: `Task_8_Merge_GroupBy_Pivot.py`

```
import pandas as pd

print("=" * 75)
```

```

print("          NUMPY & PANDAS FOR DATA ANALYTICS - TASK 8")
print("          MERGE, CONCATENATE, GROUPBY & PIVOT")
print("=" * 75)

# -----
# 1. CREATE STUDENT DETAILS DATASET
# -----

student_details = pd.DataFrame({
    "Student_ID": [
        "S101", "S102", "S103", "S104", "S105"
    ],
    "Name": [
        "Student A", "Student B", "Student C",
        "Student D", "Student E"
    ],
    "Branch": [
        "AI & DS", "CSE", "AI & DS", "ECE", "CSE"
    ]
})

# -----
# 2. CREATE PERFORMANCE DATASET
# -----

performance = pd.DataFrame({
    "Student_ID": [
        "S101", "S102", "S103", "S104", "S105"
    ],
    "Maths": [72, 85, 91, 68, 77],
    "Reading": [78, 82, 90, 70, 80],
    "Writing": [75, 88, 94, 65, 76]
})

print("\n1. STUDENT DETAILS")
print("-" * 75)
print(student_details)

print("\n2. STUDENT PERFORMANCE")
print("-" * 75)
print(performance)

# -----

```

```

# 3. MERGE DATASETS
# -----

print("\n3. MERGING DATASETS")
print("-" * 75)

merged_df = pd.merge(
    student_details,
    performance,
    on="Student_ID"
)

print(merged_df)

# -----
# 4. CONCATENATING DATA
# -----

print("\n4. CONCATENATING DATA")
print("-" * 75)

additional_students = pd.DataFrame({
    "Student_ID": ["S106", "S107"],
    "Name": ["Student F", "Student G"],
    "Branch": ["AI & DS", "ECE"]
})

concatenated_df = pd.concat(
    [student_details, additional_students],
    ignore_index=True
)

print(concatenated_df)

# -----
# 5. GROUPBY - AVERAGE MATHS SCORE BY BRANCH
# -----

print("\n5. GROUPBY - AVERAGE MATHS SCORE BY BRANCH")
print("-" * 75)

branch_average = merged_df.groupby(

```

```

        "Branch"
    )["Maths"].mean()

print(branch_average)

# -----
# 6. GROUPBY - AVERAGE OF ALL SUBJECTS BY BRANCH
# -----

print("\n6. GROUPBY - SUBJECT AVERAGES BY BRANCH")
print("-" * 75)

subject_average = merged_df.groupby(
    "Branch"
)[["Maths", "Reading", "Writing"]].mean()

print(subject_average)

# -----
# 7. PIVOT TABLE
# -----

print("\n7. PIVOT TABLE - AVERAGE MARKS BY BRANCH")
print("-" * 75)

pivot_table = pd.pivot_table(
    merged_df,
    values=["Maths", "Reading", "Writing"],
    index="Branch",
    aggfunc="mean"
)

print(pivot_table)

# -----
# 8. PIVOT TABLE BY BRANCH AND STUDENT
# -----

print("\n8. PIVOT TABLE - STUDENT PERFORMANCE")
print("-" * 75)

student_pivot = pd.pivot_table(

```

```
merged_df,  
values="Maths",  
index="Branch",  
columns="Name",  
aggfunc="mean"  
)  
  
print(student_pivot)  
  
print("\n" + "-" * 75)  
print("Merge, concatenate, GroupBy and Pivot Table operations")  
print("completed successfully.")  
print("=" * 75)
```

5. Program Output

```
(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_8_Merge_GroupBy_Pivot.py
```

```
=====
NUMPY & PANDAS FOR DATA ANALYTICS - TASK 8
MERGE, CONCATENATE, GROUPBY & PIVOT
=====
```

```
1. STUDENT DETAILS
```

```
-----
Student_ID  Name  Branch
0      S101 Student A  AI & DS
1      S102 Student B    CSE
2      S103 Student C  AI & DS
3      S104 Student D    ECE
4      S105 Student E    CSE
```

```
2. STUDENT PERFORMANCE
```

```
-----
Student_ID  Maths  Reading  Writing
0      S101     72      78      75
1      S102     85      82      88
2      S103     91      90      94
3      S104     68      70      65
4      S105     77      80      76
```

```
3. MERGING DATASETS
```

```
-----
Student_ID  Name  Branch  Maths  Reading  Writing
0      S101 Student A  AI & DS     72      78      75
1      S102 Student B    CSE     85      82      88
2      S103 Student C  AI & DS     91      90      94
3      S104 Student D    ECE     68      70      65
4      S105 Student E    CSE     77      80      76
```

4. CONCATENATING DATA

```
-----  
  Student_ID      Name      Branch  
0      S101  Student A  AI & DS  
1      S102  Student B      CSE  
2      S103  Student C  AI & DS  
3      S104  Student D      ECE  
4      S105  Student E      CSE  
5      S106  Student F  AI & DS  
6      S107  Student G      ECE
```

5. GROUPBY - AVERAGE MATHS SCORE BY BRANCH

```
-----  
Branch  
AI & DS      81.5  
CSE          81.0  
ECE          68.0  
Name: Maths, dtype: float64
```

6. GROUPBY - SUBJECT AVERAGES BY BRANCH

```
-----  
      Maths  Reading  Writing  
Branch  
AI & DS    81.5      84.0    84.5  
CSE        81.0      81.0    82.0  
ECE        68.0      70.0    65.0
```

7. PIVOT TABLE - AVERAGE MARKS BY BRANCH

	Maths	Reading	Writing
Branch			
AI & DS	81.5	84.0	84.5
CSE	81.0	81.0	82.0
ECE	68.0	70.0	65.0

8. PIVOT TABLE - STUDENT PERFORMANCE

Name	Student A	Student B	Student C	Student D	Student E
Branch					
AI & DS	72.0	NaN	91.0	NaN	NaN
CSE	NaN	85.0	NaN	NaN	77.0
ECE	NaN	NaN	NaN	68.0	NaN

Merge, concatenate, GroupBy and Pivot Table operations completed successfully.

```
(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics>
```

6. Result

The datasets were successfully combined using merge and concatenation operations. GroupBy and Pivot Table techniques were then used to generate branch-wise performance summaries.

TASK 9 – EXPORTING DATA

1. Objective

To export processed analytical data into a CSV file and verify the exported result.

2. Concepts Demonstrated

- DataFrame processing
- Calculated columns

- Row-wise average
- Sorting
- to_csv()
- File verification

3. Implementation

The student dataset was processed by calculating an average score for each student. The records were then sorted according to the average score and exported to:

output_files/processed_students.csv

The exported file was read again to verify the saved data.

4. Source Code

File: Task_9_Export_Data.py

```
import pandas as pd
import os

print("=" * 70)
print("      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 9")
print("      EXPORTING DATA")
print("=" * 70)

# -----
# 1. READ DATASET
# -----

file_path = "datasets/students.csv"

df = pd.read_csv(file_path)

print("\n1. ORIGINAL DATASET")
print("-" * 70)
print(df)

# -----
# 2. PROCESS THE DATA
```

```

# -----

print("\n2. PROCESSING DATA")
print("-" * 70)

processed_df = df[
    ["Student_ID", "Name", "Branch", "Maths", "Reading", "Writing"]
].copy()

# Calculate average score
processed_df["Average"] = processed_df[
    ["Maths", "Reading", "Writing"]
].mean(axis=1)

# Sort by average score
processed_df = processed_df.sort_values(
    by="Average",
    ascending=False
)

print("Processed Dataset:")
print(processed_df)

# -----
# 3. CREATE OUTPUT DIRECTORY
# -----

output_directory = "output_files"

os.makedirs(
    output_directory,
    exist_ok=True
)

# -----
# 4. EXPORT DATA TO CSV
# -----

output_file = os.path.join(
    output_directory,
    "processed_students.csv"
)

```

```

processed_df.to_csv(
    output_file,
    index=False
)

print("\n3. DATA EXPORT")
print("-" * 70)
print(f"File exported successfully:")
print(output_file)

# -----
# 5. VERIFY EXPORTED FILE
# -----

print("\n4. VERIFYING EXPORTED FILE")
print("-" * 70)

if os.path.exists(output_file):

    print("Export Status : SUCCESS")
    print(f"File Name      : {output_file}")
    print(
        f"File Size       : {os.path.getsize(output_file)} bytes"
    )
else:
    print("Export Status : FAILED")

# -----
# 6. READ EXPORTED FILE
# -----

print("\n5. READING EXPORTED FILE")
print("-" * 70)

verified_df = pd.read_csv(output_file)

print(verified_df)

print("\n" + "-" * 70)
print("Data exporting and verification completed successfully.")
print("=" * 70)

```

6. Program Output

```
(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_9_Export_Data.py

=====
NUMPY & PANDAS FOR DATA ANALYTICS - TASK 9
EXPORTING DATA
=====

1. ORIGINAL DATASET
-----
Student_ID  Name  Branch  Gender  Maths  Reading  Writing
0    S101  Student A  AI & DS  Female  72      78      75
1    S102  Student B    CSE  Male   85      82      88
2    S103  Student C  AI & DS  Female  91      90      94
3    S104  Student D    ECE  Male   68      70      65
4    S105  Student E    CSE  Female  77      80      76
5    S106  Student F  AI & DS  Male   88      85      90
6    S107  Student G    ECE  Female  95      92      96
7    S108  Student H    CSE  Male   81      79      84
8    S109  Student I  AI & DS  Female  74      76      72
9    S110  Student J    ECE  Male   89      87      91

2. PROCESSING DATA
-----
Processed Dataset:
Student_ID  Name  Branch  Maths  Reading  Writing  Average
6    S107  Student G    ECE   95      92      96  94.333333
2    S103  Student C  AI & DS   91      90      94  91.666667
9    S110  Student J    ECE   89      87      91  89.000000
5    S106  Student F  AI & DS   88      85      90  87.666667
1    S102  Student B    CSE   85      82      88  85.000000
7    S108  Student H    CSE   81      79      84  81.333333
4    S105  Student E    CSE   77      80      76  77.666667
0    S101  Student A  AI & DS   72      78      75  75.000000
8    S109  Student I  AI & DS   74      76      72  74.000000
3    S104  Student D    ECE   68      70      65  67.666667

3. DATA EXPORT
-----
File exported successfully:
output_files\processed_students.csv

4. VERIFYING EXPORTED FILE
-----
Export Status : SUCCESS
File Name      : output_files\processed_students.csv
File Size      : 488 bytes

5. READING EXPORTED FILE
-----
Student_ID  Name  Branch  Maths  Reading  Writing  Average
0    S107  Student G    ECE   95      92      96  94.333333
1    S103  Student C  AI & DS   91      90      94  91.666667
2    S110  Student J    ECE   89      87      91  89.000000
3    S106  Student F  AI & DS   88      85      90  87.666667
4    S102  Student B    CSE   85      82      88  85.000000
5    S108  Student H    CSE   81      79      84  81.333333
6    S105  Student E    CSE   77      80      76  77.666667
7    S101  Student A  AI & DS   72      78      75  75.000000
8    S109  Student I  AI & DS   74      76      72  74.000000
9    S104  Student D    ECE   68      70      65  67.666667

-----
Data exporting and verification completed successfully.
=====
(venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics>
```

7. Result

The processed student dataset was successfully exported to a CSV file and verified by reading the exported file.

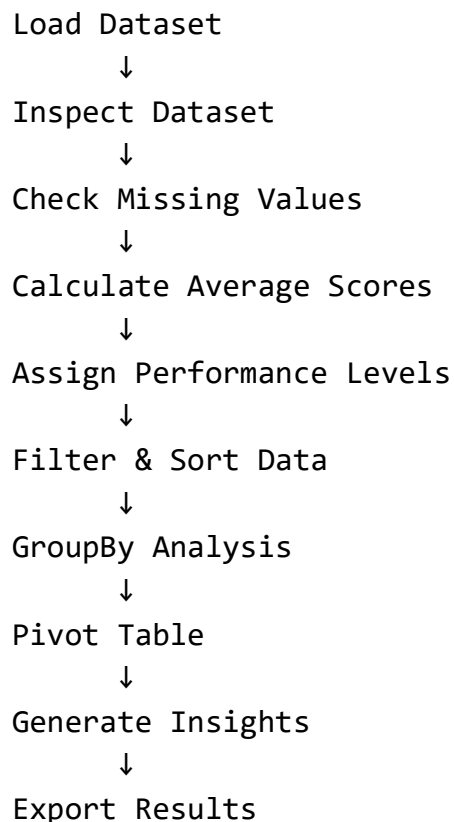
TASK 10 — MINI DATA ANALYSIS PROJECT

Student Performance Data Analysis Using Pandas

1. Objective

To apply the concepts learned throughout Week 2 to perform a complete data analytics workflow on a student performance dataset.

2. Project Workflow



3. Concepts Demonstrated

- CSV data loading
- Dataset inspection
- Missing-value analysis
- Calculated columns
- Conditional classification
- Filtering
- Sorting
- GroupBy
- Pivot Table
- Statistical analysis
- CSV export

4. Implementation

The student performance dataset was loaded using Pandas and inspected to understand its structure.

An **Average** column was calculated using Maths, Reading, and Writing scores. Students were then classified into performance levels based on their average scores.

The dataset was filtered and sorted to identify high-performing students.

Branch-wise performance was analyzed using `groupby()`, while Pivot Table operations were used to summarize subject-level performance.

Finally, the complete analytical dataset was exported to:

`output_files/student_performance_analysis.csv`

5. Source Code

File: `Task_10_Mini_Data_Analysis.py`

```
"""
Week 2 Internship: NumPy & Pandas for Data Analytics
Task 10: Mini Data Analysis Project
```

Project:

Student Performance Data Analysis using Pandas

This project demonstrates a complete data analytics workflow using a student performance dataset.

"""

```
import pandas as pd
import os
```

```
print("=" * 75)
print("      NUMPY & PANDAS FOR DATA ANALYTICS - TASK 10")
print("      MINI STUDENT PERFORMANCE ANALYSIS")
print("=" * 75)
```

```
# -----
# 1. LOAD DATASET
# -----
```

```
file_path = "datasets/students.csv"
```

```
df = pd.read_csv(file_path)
```

```
print("\n1. DATASET LOADED")
print("-" * 75)
print(df)
```

```
# -----
# 2. INSPECT DATASET
# -----
```

```
print("\n2. DATASET INSPECTION")
print("-" * 75)
```

```
print(f"Number of Rows      : {df.shape[0]}")
print(f"Number of Columns   : {df.shape[1]}")
print(f"Dataset Shape        : {df.shape}")
```

```
print("\nColumn Names:")
print(list(df.columns))
```

```
print("\nData Types:")
```

```

print(df.dtypes)

# -----
# 3. CHECK MISSING VALUES
# -----

print("\n3. MISSING VALUE ANALYSIS")
print("-" * 75)

missing_values = df.isnull().sum()

print(missing_values)

total_missing = missing_values.sum()

print(f"\nTotal Missing Values: {total_missing}")

# -----
# 4. CREATE AVERAGE SCORE
# -----

print("\n4. CALCULATING AVERAGE SCORES")
print("-" * 75)

subject_columns = ["Maths", "Reading", "Writing"]

df["Average"] = df[subject_columns].mean(axis=1)

print(df[
    ["Student_ID", "Name", "Branch", "Average"]
])

# -----
# 5. ASSIGN PERFORMANCE LEVEL
# -----

print("\n5. ASSIGNING PERFORMANCE LEVEL")
print("-" * 75)

def performance_level(average):

```

```

        if average >= 90:
            return "Excellent"
        elif average >= 80:
            return "Good"
        elif average >= 70:
            return "Average"
        else:
            return "Needs Improvement"

df["Performance"] = df["Average"].apply(
    performance_level
)

print(df[
    ["Student_ID", "Name", "Average", "Performance"]
])

# -----
# 6. FILTER HIGH-PERFORMING STUDENTS
# -----

print("\n6. HIGH-PERFORMING STUDENTS")
print("-" * 75)

high_performers = df[df["Average"] >= 80]

print(
    high_performers[
        ["Student_ID", "Name", "Branch", "Average", "Performance"]
    ]
)

# -----
# 7. SORT STUDENTS BY AVERAGE SCORE
# -----

print("\n7. STUDENTS SORTED BY AVERAGE SCORE")
print("-" * 75)

sorted_students = df.sort_values(
    by="Average",

```

```

        ascending=False
    )

    print(
        sorted_students[
            ["Student_ID", "Name", "Branch", "Average"]
        ]
    )

# -----
# 8. BRANCH-WISE ANALYSIS USING GROUPBY
# -----

print("\n8. BRANCH-WISE PERFORMANCE ANALYSIS")
print("-" * 75)

branch_analysis = df.groupby("Branch")[
    ["Maths", "Reading", "Writing", "Average"]
].mean()

print(branch_analysis.round(2))

# -----
# 9. PIVOT TABLE
# -----

print("\n9. PIVOT TABLE")
print("-" * 75)

pivot = pd.pivot_table(
    df,
    values=["Maths", "Reading", "Writing"],
    index="Branch",
    aggfunc="mean"
)

print(pivot.round(2))

# -----
# 10. OVERALL STATISTICS
# -----

```

```

print("\n10. OVERALL PERFORMANCE")
print("-" * 75)

overall_average = df["Average"].mean()
highest_average = df["Average"].max()
lowest_average = df["Average"].min()

top_student = df.loc[
    df["Average"].idxmax()
]

print(f"Overall Average Score : {overall_average:.2f}")
print(f"Highest Average Score : {highest_average:.2f}")
print(f"Lowest Average Score : {lowest_average:.2f}")

print(
    f"Top Student          : "
    f"{top_student['Name']}"
)

print(
    f"Top Student Average   : "
    f"{top_student['Average']:.2f}"
)

# -----
# 11. SUBJECT-WISE AVERAGE
# -----

print("\n11. SUBJECT-WISE AVERAGE")
print("-" * 75)

subject_averages = df[subject_columns].mean()

print(subject_averages.round(2))

# -----
# 12. EXPORT ANALYSIS RESULT
# -----

print("\n12. EXPORTING ANALYSIS RESULT")
print("-" * 75)

```

```

output_directory = "output_files"

os.makedirs(
    output_directory,
    exist_ok=True
)

output_file = os.path.join(
    output_directory,
    "student_performance_analysis.csv"
)

df.to_csv(
    output_file,
    index=False
)

print(f"Analysis exported to: {output_file}")

# -----
# 13. FINAL INSIGHTS
# -----

print("\n13. KEY INSIGHTS")
print("-" * 75)

best_subject = subject_averages.idxmax()
best_subject_score = subject_averages.max()

best_branch = branch_analysis["Average"].idxmax()
best_branch_score = branch_analysis["Average"].max()

high_performer_count = len(high_performers)

print(
    f"1. The overall average student score is "
    f"{overall_average:.2f}."
)

print(
    f"2. {high_performer_count} students have an "

```

```

        f"average score of 80 or above."
    )

    print(
        f"3. {best_subject} has the highest subject "
        f"average of {best_subject_score:.2f}."
    )

    print(
        f"4. {best_branch} has the highest branch-wise "
        f"average of {best_branch_score:.2f}."
    )

    print(
        f"5. {top_student['Name']} achieved the highest "
        f"overall average of {top_student['Average']:.2f}."
    )

    print("\n" + "-" * 75)
    print("Mini data analysis project completed successfully.")
    print("=" * 75)

```

6. Program Output

```

> (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> python source_code\Task_10_Mini_Data_Analysis.py
=====
          NUMPY & PANDAS FOR DATA ANALYTICS - TASK 10
          MINI STUDENT PERFORMANCE ANALYSIS
=====

1. DATASET LOADED
-----

```

	Student_ID	Name	Branch	Gender	Maths	Reading	Writing
0	S101	Student A	AI & DS	Female	72	78	75
1	S102	Student B	CSE	Male	85	82	88
2	S103	Student C	AI & DS	Female	91	90	94
3	S104	Student D	ECE	Male	68	70	65
4	S105	Student E	CSE	Female	77	80	76
5	S106	Student F	AI & DS	Male	88	85	90
6	S107	Student G	ECE	Female	95	92	96
7	S108	Student H	CSE	Male	81	79	84
8	S109	Student I	AI & DS	Female	74	76	72
9	S110	Student J	ECE	Male	89	87	91

2. DATASET INSPECTION

Number of Rows : 10
Number of Columns : 7
Dataset Shape : (10, 7)

Column Names:

['Student_ID', 'Name', 'Branch', 'Gender', 'Maths', 'Reading', 'Writing']

Data Types:

Student_ID str
Name str
Branch str
Gender str
Maths int64
Reading int64
Writing int64
dtype: object

3. MISSING VALUE ANALYSIS

Student_ID 0
Name 0
Branch 0
Gender 0
Maths 0
Reading 0
Writing 0
dtype: int64

Total Missing Values: 0

4. CALCULATING AVERAGE SCORES

	Student_ID	Name	Branch	Average
0	S101	Student A	AI & DS	75.000000
1	S102	Student B	CSE	85.000000
2	S103	Student C	AI & DS	91.666667
3	S104	Student D	ECE	67.666667
4	S105	Student E	CSE	77.666667
5	S106	Student F	AI & DS	87.666667
6	S107	Student G	ECE	94.333333
7	S108	Student H	CSE	81.333333
8	S109	Student I	AI & DS	74.000000
9	S110	Student J	ECE	89.000000

5. ASSIGNING PERFORMANCE LEVEL

	Student_ID	Name	Average	Performance
0	S101	Student A	75.000000	Average
1	S102	Student B	85.000000	Good
2	S103	Student C	91.666667	Excellent
3	S104	Student D	67.666667	Needs Improvement
4	S105	Student E	77.666667	Average
5	S106	Student F	87.666667	Good
6	S107	Student G	94.333333	Excellent
7	S108	Student H	81.333333	Good
8	S109	Student I	74.000000	Average
9	S110	Student J	89.000000	Good

6. HIGH-PERFORMING STUDENTS

	Student_ID	Name	Branch	Average	Performance
1	S102	Student B	CSE	85.000000	Good
2	S103	Student C	AI & DS	91.666667	Excellent
5	S106	Student F	AI & DS	87.666667	Good
6	S107	Student G	ECE	94.333333	Excellent
7	S108	Student H	CSE	81.333333	Good
9	S110	Student J	ECE	89.000000	Good

7. STUDENTS SORTED BY AVERAGE SCORE

	Student_ID	Name	Branch	Average
6	S107	Student G	ECE	94.333333
2	S103	Student C	AI & DS	91.666667
9	S110	Student J	ECE	89.000000
5	S106	Student F	AI & DS	87.666667
1	S102	Student B	CSE	85.000000
7	S108	Student H	CSE	81.333333
4	S105	Student E	CSE	77.666667
0	S101	Student A	AI & DS	75.000000
8	S109	Student I	AI & DS	74.000000
3	S104	Student D	ECE	67.666667

8. BRANCH-WISE PERFORMANCE ANALYSIS

	Maths	Reading	Writing	Average
Branch				
AI & DS	81.25	82.25	82.75	82.08
CSE	81.00	80.33	82.67	81.33
ECE	84.00	83.00	84.00	83.67

9. PIVOT TABLE

	Maths	Reading	Writing
Branch			
AI & DS	81.25	82.25	82.75
CSE	81.00	80.33	82.67
ECE	84.00	83.00	84.00

10. OVERALL PERFORMANCE

Overall Average Score : 82.33
Highest Average Score : 94.33
Lowest Average Score : 67.67
Top Student : Student G
Top Student Average : 94.33

```
11. SUBJECT-WISE AVERAGE
-----
Maths      82.0
Reading    81.9
Writing    83.1
dtype: float64

12. EXPORTING ANALYSIS RESULT
-----
Analysis exported to: output_files\student_performance_analysis.csv

13. KEY INSIGHTS
-----
1. The overall average student score is 82.33.
2. 6 students have an average score of 80 or above.
3. Writing has the highest subject average of 83.10.
4. ECE has the highest branch-wise average of 83.67.
5. Student G achieved the highest overall average of 94.33.

-----
Mini data analysis project completed successfully.
=====
❖ (venv) PS D:\projects\InternNova_Internship\Week_2_Python_Data_Analytics> 
```

7. Key Findings

The analysis generated several useful observations from the student performance dataset:

- Student average scores were calculated successfully.
- Students with average scores of **80 or above** were identified.
- Subject-wise average performance was calculated.
- Branch-wise average performance was obtained using GroupBy.
- The highest-performing student was identified.
- The processed analytical dataset was exported for further use.

OVERALL LEARNING OUTCOMES

Through the Week 2 internship tasks, I developed practical knowledge of NumPy and Pandas for data analytics.

Technical Skills

- NumPy array creation and manipulation
- Array indexing and slicing
- Array reshaping
- Numerical calculations
- Statistical analysis
- Pandas Series and DataFrames
- CSV data processing
- Data inspection
- Data filtering and sorting
- Missing-value handling
- Dataset merging and concatenation
- GroupBy analysis
- Pivot Tables
- Data export

Analytical Skills

- Understanding dataset structure
- Identifying data-quality issues
- Transforming raw data
- Extracting meaningful information
- Comparing groups
- Generating analytical summaries
- Preparing processed datasets for further use

WEEK 2 LEARNING REFLECTION

LEARNING REFLECTION

During Week 2, I progressed from basic Python programming to practical data manipulation and analysis using NumPy and Pandas.

Working with NumPy helped me understand how numerical data can be represented and manipulated efficiently through arrays. Pandas provided practical tools for working with structured datasets, including filtering, sorting, cleaning, aggregation, and transformation.

The mini project helped me understand how individual data-processing techniques can be combined into a complete analytics workflow.

This week also strengthened my understanding of the importance of **data inspection and cleaning before performing analysis**.

CONCLUSION

The Week 2 internship task on **NumPy and Pandas for Data Analytics** was successfully completed through a series of practical exercises and a mini data analysis project.

The tasks provided hands-on experience in numerical computing, structured data manipulation, data cleaning, aggregation, transformation, and export.

The final **Student Performance Data Analysis** project brought together the concepts learned throughout the week and demonstrated how Python libraries can be used to convert raw data into structured and meaningful analytical information.

Overall, Week 2 strengthened my practical foundation in **Python-based data analytics** and prepared me for more advanced data science and machine learning workflows.

PROJECT FILES & SUBMISSION

Project Structure

Week_2_Python_Data_Analytics/

- |
- |— source_code/
 - |— Task_1_NumPy_Arrays.py
 - |— Task_2_Indexing_Slicing_Reshaping.py
 - |— Task_3_NumPy_Math_Statistics.py
 - |— Task_4_Pandas_Series_DataFrame.py
 - |— Task_5_Read_Inspect_Data.py
 - |— Task_6_Filter_Sort_Data.py
 - |— Task_7_Missing_Values.py
 - |— Task_8_Merge_GroupBy_Pivot.py
 - |— Task_9_Export_Data.py
 - |— Task_10_Mini_Data_Analysis.py
- |
- |— datasets/
 - |— students.csv
- |
- |— output_files/
 - |— processed_students.csv
 - |— student_performance_analysis.csv
- |
- |— screenshots/
- |
- |— report/
 - |— Week_2_Internship_Report.pdf